

Notes on Policy Iteration

Luis Mendez

POLICY ITERATION

Policy iteration is a dynamic programming algorithm for computing the optimal policy π^* of a Markov Decision Process, given a full model of the environment (transition probabilities $P(s'|s, a)$ and reward function $R(s, a, s')$). Instead of iterating on the value function directly like value iteration does, it maintains an explicit policy and alternates between two steps: evaluating how good the current policy is, and then making it greedy with respect to that evaluation. It repeats this until the policy stops changing.

Bellman Expectation Equation

Where value iteration is built on the Bellman *optimality* equation (which has a max in it), policy evaluation is built on the Bellman *expectation* equation, which fixes the policy and so has no max:

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^\pi(s')].$$

For a deterministic policy $\pi(s)$ this simplifies to

$$V^\pi(s) = \sum_{s'} P(s'|s, \pi(s)) [R(s, \pi(s), s') + \gamma V^\pi(s')].$$

Because the policy is fixed, this is a system of $|S|$ linear equations in $|S|$ unknowns, which is the key structural difference from the optimality equation. The max is what makes the Bellman optimality equation nonlinear and forces us to iterate, without it we could in principle just solve the system directly.

Step 1: Policy Evaluation

Given a fixed policy π , we want V^π , the value of every state if we follow π forever. There are two ways to get it:

Solve it directly. Writing the system in matrix form with $V^\pi \in \mathbb{R}^{|S|}$, the reward vector R^π , and the transition matrix P^π induced by the policy,

$$V^\pi = R^\pi + \gamma P^\pi V^\pi \implies V^\pi = (I - \gamma P^\pi)^{-1} R^\pi.$$

The inverse exists whenever $\gamma < 1$. This gives V^π exactly, but costs $O(|S|^3)$, which is impractical past a few thousand states.

Solve it iteratively. More commonly we just sweep the Bellman expectation backup until it converges:

$$V_{k+1}(s) = \sum_{s'} P(s'|s, \pi(s)) [R(s, \pi(s), s') + \gamma V_k(s')], \quad \forall s.$$

This is the same style of update as value iteration but with the action fixed by π rather than maximized over. It is also a contraction with factor γ , so it converges to V^π from any

starting point. A good trick is to warm start each evaluation from the previous iteration's value function, since the policy usually changes only slightly, V^π is usually close to what we already had, and evaluation converges in far fewer sweeps.

Step 2: Policy Improvement

Once we have V^π , we build a new policy that acts greedily with respect to it:

$$\pi'(s) = \arg \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^\pi(s')] = \arg \max_a Q^\pi(s, a)$$

Note the asymmetry: we evaluate under π , but we improve by considering *all* actions, not just the one π would take. If for some state a different action has a higher $Q^\pi(s, a)$ than the action π currently picks, switching to it can only help.

Notation

It is common to write the model as a single joint distribution $p(s', r | s, a)$, the probability of landing in state s' and receiving reward r after taking action a in state s . The backup then sums over both:

$$\sum_{s', r} p(s', r | s, a) [r + \gamma V(s')].$$

This is equivalent to the $\sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V(s')]$ form used above, it just keeps the reward as part of the random outcome instead of folding it into an expected-reward function. The joint form is the more general one (it allows the reward itself to be stochastic given (s, a, s')), so the algorithm below is written that way.

Algorithm 1 Policy Iteration

```

1: Input: tolerance  $\theta > 0$ , discount factor  $\gamma$ 
2: Initialize  $V(s)$  and  $\pi(a|s)$  arbitrarily
3: while policy-stable = false do
    Policy Evaluation:
4:   while  $\Delta > \theta$  do
5:      $\Delta \leftarrow 0$ 
6:     for  $s \in S$  do
7:        $v \leftarrow V(s)$ 
8:        $V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$ 
9:        $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
10:    end for
11:  end while
    Policy Improvement:
12:  policy-stable  $\leftarrow$  true
13:  for  $s \in S$  do
14:    old-action  $\leftarrow \pi(s)$ 
15:     $\pi(s) \leftarrow \arg \max_{a \in A(s)} \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$ 
16:    if old-action  $\neq \pi(s)$  then
17:      policy-stable  $\leftarrow$  false
18:    end if
19:  end for
20: end while
21: Output: optimal policy  $\pi(a|s)$  and state values  $V(s)$ 

```

A few things worth noticing in this structure:

- **The evaluation step keeps $\pi(a|s)$ general.** Writing $\sum_a \pi(a|s) (\cdot)$ handles a stochastic policy, we average the backup over the actions the policy might take. If π is deterministic that inner sum collapses to the single term $a = \pi(s)$, which is the simplified form given earlier.
- **But the improvement step assigns $\pi(s)$ a single action.** The $\arg \max$ always yields a deterministic policy, so after the first improvement pass the policy is deterministic even if it started stochastic. This is fine, a finite MDP always has a deterministic optimal policy.
- **Δ must be reset before each evaluation loop.** The `while  $\Delta > \theta$` test reads Δ before the body sets it, so Δ needs to be initialized to something larger than θ (or the loop written as repeat-until) for the evaluation to run at all on the first pass.
- **The outer loop is driven by policy-stable, not by values.** Convergence is declared when an entire improvement sweep changes no action, not when V stops moving. That is the stopping condition that gives the exactness guarantee described below.

When checking whether the policy is stable, ties matter: if two actions have equal Q values and the $\arg \max$ picks arbitrarily between them, the policy can flip back and forth forever without actually improving. The usual fix is to only mark the policy unstable if the new action's value is strictly

better, or to break ties consistently (e.g., by lowest action index).

The Policy Improvement Theorem

The guarantee that makes this work is the policy improvement theorem: if π' is greedy with respect to V^π , then

$$Q^\pi(s, \pi'(s)) \geq V^\pi(s) \quad \forall s \implies V^{\pi'}(s) \geq V^\pi(s) \quad \forall s.$$

That is, being greedy for a single step and then following the old policy is at least as good as following the old policy from the start, and by unrolling that argument, committing to the greedy policy everywhere is at least as good in every state. So each improvement step produces a policy that is weakly better everywhere, never worse in any state.

Furthermore, if the improvement step changes nothing ($\pi' = \pi$), then V^π satisfies

$$V^\pi(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^\pi(s')],$$

which is exactly the Bellman optimality equation, so $V^\pi = V^*$ and π is optimal. Convergence is therefore not just approximate here: when policy iteration halts, it has found an exactly optimal policy.

Why it Terminates

For a finite MDP there are only $|A|^{|S|}$ deterministic policies. Each iteration produces a strictly better policy (or halts), and we can never revisit a policy we have already left because values are monotonically non-decreasing. So the algorithm must terminate after finitely many iterations. In practice the bound is wildly pessimistic, policy iteration typically converges in a handful of iterations, often far fewer than the number of sweeps value iteration needs.

Modified (Truncated) Policy Iteration

Running policy evaluation to full convergence at every iteration is wasteful, the early value estimates are only being used to decide which action is best, and the ranking of actions usually settles long before the values do. Modified policy iteration just caps evaluation at k sweeps instead of running to θ :

- $k = \infty$ recovers standard policy iteration.
- $k = 1$ recovers value iteration, one evaluation sweep immediately followed by a greedy improvement is algebraically the same as a single Bellman optimality backup.
- Intermediate k (say 3 to 10) is often faster than either extreme in practice.

This is the cleanest way to see that value iteration and policy iteration are not two separate algorithms so much as two ends of one spectrum, controlled by how much evaluation you do between improvements.

Generalized Policy Iteration

The broader pattern, evaluation pushing V toward V^π and improvement pushing π toward greedy with respect to V , is called generalized policy iteration, and nearly every RL algorithm is an instance of it. The two processes compete (making the policy greedy invalidates the value function, and re-evaluating the value function makes the policy no longer greedy) but they converge together on the single point where both conditions hold at once, which is V^* and π^* . Monte Carlo control, SARSA, and Q-learning are all this same loop with the evaluation step replaced by sampled experience instead of a known model.

Policy Iteration vs. Value Iteration

- **Cost per iteration:** policy iteration is more expensive, each iteration requires a full evaluation (either an $O(|S|^3)$ solve or many sweeps) plus an improvement pass. Value iteration does one $O(|S|^2|A|)$ backup per sweep.
- **Number of iterations:** policy iteration needs far fewer, often under ten, because each improvement step uses a nearly exact value function. Value iteration needs many more but each is cheap.
- **Termination:** policy iteration has a clean stopping condition (the policy stopped changing) and returns an exactly optimal policy. Value iteration only converges in the limit and stops on a threshold θ , though its greedy policy is usually optimal well before V converges.
- **When to prefer which:** policy iteration tends to win when $|A|$ is large (the max over actions is expensive, and value iteration pays it every sweep) or when γ is close to 1 (value iteration's contraction is slow, but policy iteration's convergence does not degrade the same way). Value iteration tends to win when the state space is large enough that a full evaluation is unaffordable.

Requirements

Like all dynamic programming methods for MDPs, policy iteration assumes a full model of the environment (P and R known) and a state space small enough to sweep exhaustively. When the model is unknown, the evaluation step is replaced by sampling returns from experience (Monte Carlo) or by bootstrapped sampled backups (temporal difference methods), which is exactly how the generalized policy iteration pattern carries over into model-free control.